

МЕТОДИЧЕСКИЕ УКАЗАНИЯ ДЛЯ СТУДЕНТОВ

1. Необходимые знания

- Python (базовый).
- Основы PyTorch (тензоры, авто дифференцирование, nn.Module).
- Архитектуры RNN/LSTM/GRU.

2. Порядок выполнения работы

1. Выбор текстового корпуса:

- Выберите предложенный в задании текст (или используйте свой), размер текстового файла 50-100Кб.
Для отладки кода используйте синтетический текст, заданный в шаблоне кода.
- При использовании собственного текста сохраните его в файл .txt в кодировке UTF-8,
удалите лишние пробелы и переносы строк (по желанию).

2. Создание алфавита и кодирование:

Для получения уникальных символов текста используйте фрагмент кода:

```
# Получение уникальных символов
chars = sorted(list(set(text)))
char_to_idx = {ch: i for i, ch in enumerate(chars)}
idx_to_char = {i: ch for i, ch in enumerate(chars)}
vocab_size = len(chars)
print(f"Размер алфавита: {vocab_size}")
```

3. Формирование последовательностей:

Для формирования входной и выходной (целевой) обучающих последовательностей используйте псевдокод:

```
inputs, targets = [], []
# ЗАДАНИЕ: Реализуйте создание последовательностей
for i in range(len(text) - seq_length):
    inputs.append(...)
    targets.append(...)
```

4. Реализация модели:

- Используйте nn.Embedding для встраивания символов.
- Используйте nn.LSTM с batch_first=True.
- Добавьте nn.Linear для проекции на словарь.
- Прямой проход должен возвращать логиты и новое скрытое состояние.

5. Обучение модели:

- Используйте DataLoader для загрузки батчей данных.
- Отслеживайте потери на каждой эпохе.
- Постройте график потерь.

6. Реализация функции генерации:

Для генерации выходной (целевой) последовательности с параметром `temperature` используйте шаблон псевдокода:

```
def generate_text(model, start_string, length, temperature=1.0,
seq_length=100):
    """
    Генерация текста с использованием обученной модели.

    model.eval()

    # 1. Преобразование начальной строки в индексы
    # 2. Цикл генерации символов
    # 3. Sampling с температурой
    # 4. Преобразование индексов в текст

    return generated_text
```

7. Генерация примеров:

- Сгенерируйте текст с температурами 0.5, 1.0, 1.5.
- Сравните результаты.

8. Оформление отчёта.

В Git-репозиторий (или в виде архива) должны быть загружены:

1. **Jupyter Notebook** со всеми выполненными ячейками (код, графики, результаты).
2. **Текстовый файл** с корпусом (или ссылка на источник).
3. **Отчёт** (PDF или Markdown), содержащий:
 - Описание текстового корпуса (размер, язык, источник).
 - График потерь с комментариями о сходимости.
 - Примеры сгенерированных текстов для разных температур.
 - Анализ качества: сравнение температур, осмысленность, грамматика.
 - Сравнение LSTM и GRU.
 - Исследование влияния длины последовательности.

3. Рекомендуемые гиперпараметры

Параметр	Значение	Пояснение
seq_length	100	Длина входной последовательности
batch_size	128	Размер батча
embed_size	128	Размерность встраивания символов
hidden_size	256	Размер скрытого состояния LSTM
num_layers	2	Количество слоёв LSTM
learning_rate	0.002	Скорость обучения
num_epochs	20–30	Количество эпох
temperature	0.5, 1.0, 1.5	Для генерации (сравнение)